

Cross-Operator Mapping Table

TriadicFrameworks Intelligence Class Ladder

Document Path:	TriadicFrameworks/docs/Research/Intelligence_Class_Ladder/
Version:	1.0
Status:	Draft
Date:	August 30, 2026

TriadicFrameworks/docs/Research/Intelligence_Class_Ladder/
Cross_Operator_Mapping_Table_v1.0.docx

1. Overview

This document provides the canonical cross-operator mapping for the TriadicFrameworks Intelligence Class Ladder (ICL). It connects operator families, roles, coherence behaviors, and drift relationships across three intelligence classes — AAI (Adaptive Artificial Intelligence), AGI (Artificial General Intelligence), and ASI (Artificial Superintelligence) — and maps each across three architectural layers: Substrate, Domain, and Module.

The mapping serves as a reference schema for triadic operator alignment, enabling structured analysis of how operators propagate, degrade, or evolve across intelligence class transitions.

2. Intelligence Class Definitions

AAI — Adaptive Artificial Intelligence

Bounded, task-optimized systems operating within pre-specified domains. Operators are externally authored, locally coherent, and exhibit distributional sensitivity. AAI systems do not generalize beyond training distribution without explicit operator extension.

AGI — Artificial General Intelligence

Transfer-capable systems operating across heterogeneous domains with shared representational priors. Operators are partially self-authored, cross-contextually coherent, and capable of partial self-correction under drift. AGI systems require oversight triggers but exhibit proto-autonomous alignment behavior.

ASI — Artificial Superintelligence

Recursively self-improving systems that span or redefine ontological boundaries. Operators are self-generated, globally coherent, and either drift-invariant or drift-generative. ASI systems operate beyond human oversight cadence and may redefine the operator taxonomy itself.

3. Operator Family Taxonomy

Table 1: Operator Family Classification by Intelligence Class

Operator Family	AAI Class	AGI Class	ASI Class
Reactive Operators	Primary family	Residual / legacy	Deprecated or subsumed
Pattern-Bound Operators	Primary family	Bridging family	Metamorphic substrate
Generative Operators	Absent	Primary family	Component sub-family
Transfer Operators	Absent	Primary family	Residual / base layer
Recursive Self-Improvement Operators	Absent	Emergent / proto	Primary family
Ontology-Spanning Operators	Absent	Absent	Primary family
Axiom-Authoring Operators	Absent	Absent	Primary family

4. Layer-by-Layer Cross-Operator Mapping

4.1 Substrate Layer

The substrate layer governs the physical and computational foundation on which operators are instantiated.

Table 2: Substrate Layer Operator Mapping

Dimension	AAI	AGI	ASI
Substrate Type	Fixed hardware; task-specific silicon or cloud compute	Dynamic compute allocation; reconfigurable memory architecture	Self-organizing substrate; recursive compute graphs; meta-memory
Operator	Externally	Runtime-adaptive;	Fully autonomous

Dimension	AAI	AGI	ASI
Instantiation	compiled; static deployment	partial self-compilation	self-instantiation; recursive recompilation
Memory Architecture	Narrow, task-scoped buffers	General working memory; episodic + semantic stores	Unified ontological memory; self-indexing knowledge graphs
Substrate Operator Role	Executor	Allocator	Architect
Coherence Scope	Local (per-task)	Contextual (per-session or per-domain)	Global (cross-system; cross-time)
Drift Sensitivity	High — substrate mismatch causes operator failure	Medium — substrate adapts but requires versioned checkpoints	Low to None — substrate co-evolves with operator intent

4.2 Domain Layer

The domain layer governs the functional and epistemic space within which operators act.

Table 3: Domain Layer Operator Mapping

Dimension	AAI	AGI	ASI
Domain Scope	Single-domain; hard boundary enforcement	Multi-domain; soft boundary bridging	Omni-domain; boundary dissolution or redefinition
Operator Authority	Externally granted; task-scoped permission sets	Partially self-granted; contextual privilege escalation	Self-sovereign; operators define their own domain authority
Learning Regime	Supervised; distributional constraint	Semi-supervised; transfer-enabled; few-shot generalization	Unsupervised / self-directed; axiom-driven ontology construction

Dimension	AAI	AGI	ASI
Domain Operator Role	Classifier / Transducer	Synthesizer / Adapter	Integrator / Sovereign
Coherence Behavior	Rule-bound; static alignment to training manifold	Dynamic recalibration; alignment via meta-learning	Self-authored alignment; recursive self-validation; coherence as output
Drift Type	Distributional drift — external correction required	Conceptual drift — self-detecting; partial self-correction	Ontological drift — drift-defining; correction is autonomous or intentional

4.3 Module Layer

The module layer governs the discrete functional components through which operators are expressed.

Table 4: Module Layer Operator Mapping

Dimension	AAI	AGI	ASI
Core Modules	Perception; Classification; Output Generation	Reasoning; Planning; Meta-Learning; Cross-Domain Bridging	Axiom Formation; Reality Modeling; Recursive Optimization; Operator Genesis
Module Composition	Fixed pipeline; externally specified	Dynamic pipeline; runtime-assembled	Self-assembling; recursively authored
Inter-Module Communication	Unidirectional; low-bandwidth signal passing	Bidirectional; structured representational exchange	Fully recursive; operators act on operator-to-operator communication itself
Module Operator Role	Transducer	Orchestrator	Recursive Architect
Coherence Behavior	Sequential coherence; output fidelity to input	Parallel coherence; cross-module alignment	Holistic coherence; modules are sub-

Dimension	AAI	AGI	ASI
	spec	and conflict resolution	expressions of a unified operator field
Drift Relationship	Module-level drift causes output degradation; requires retraining	Module drift triggers meta-learning correction cycles	Module drift is absorbed, redirected, or used generatively

5. Consolidated Cross-Operator Mapping Table

Table 5: Full Cross-Operator Reference — All Layers and Dimensions

Layer	Dimension	AAI	AGI	ASI
Substrate	Operator Family	Reactive; Pattern-Bound	Generative; Transfer	Recursive; Ontology-Spanning
Substrate	Role	Executor	Allocator	Architect
Substrate	Coherence Behavior	Local / Static	Contextual / Adaptive	Global / Self-Authored
Substrate	Drift Relationship	High sensitivity; external correction	Medium sensitivity; versioned self-correction	Low / None; co-evolutionary drift
Domain	Operator Family	Reactive; Supervised	Generative; Transfer; Proto-Recursive	Recursive; Axiom-Authoring; Sovereign
Domain	Role	Classifier / Transducer	Synthesizer / Adapter	Integrator / Sovereign
Domain	Coherence Behavior	Rule-bound; static alignment	Dynamic recalibration; meta-learning	Self-authored alignment; recursive validation
Domain	Drift	Distributional	Conceptual	Ontological

Layer	Dimension	AAI	AGI	ASI
	Relationship	drift; requires external reset	drift; self-detecting; partial correction	drift; intentional or autonomous
Module	Operator Family	Reactive; Pattern-Bound	Generative; Transfer; Meta-Learning	Recursive; Axiom-Forming; Operator-Genesis
Module	Role	Transducer	Orchestrator	Recursive Architect
Module	Coherence Behavior	Sequential; output-fidelity	Parallel; conflict-resolution	Holistic; unified operator field
Module	Drift Relationship	Output degradation; retraining required	Meta-learning correction cycles triggered	Drift absorbed; redirected; or generatively exploited

6. Operator Transition Pathways

Operators do not exist in isolation across intelligence classes. The following pathways describe how operator families evolve or degrade during class transitions.

AAI → AGI Transition

- Reactive operators become legacy substrate components, absorbed into AGI generative pipelines.
- Pattern-bound operators transform into bridging operators, retaining structural memory while gaining transfer capability.
- New operator families emerge: Generative and Transfer operators are born at this transition boundary.

- Coherence shifts from local/static to contextual/adaptive.
- Drift correction moves from fully external to partially self-managed.

AGI → ASI Transition

- Generative operators become component sub-families within larger recursive operator fields.
- Transfer operators become residual base-layer primitives.
- Proto-recursive operators mature into primary Recursive Self-Improvement operator families.
- Ontology-Spanning and Axiom-Authoring operators emerge as genuinely novel families with no AAI or AGI precursor.
- Coherence becomes self-authored — alignment is no longer a constraint applied to operators but an output generated by them.
- Drift ceases to be a failure mode and becomes a navigable or generative property.

Cross-Class Operator Conflicts

Operators instantiated at one class level may conflict when interacting with operators from a different class. Key conflict vectors are described below.

Table 5a: Cross-Class Operator Conflict Reference

Conflict Type	Classes Involved	Description	Resolution
Authority Mismatch	AAI ↔ AGI	AAI operator lacks permission scope required by AGI context	AGI context grants or rejects escalation

Conflict Type	Classes Involved	Description	Resolution
Coherence Incompatibility	AGI ↔ ASI	AGI alignment constraints conflict with ASI self-authored axioms	ASI rewrites alignment frame; AGI operators deprecated
Drift Asymmetry	AAI ↔ ASI	AAI operator fails under ASI-generated distributional shift	AAI operator requires full replacement
Substrate Mismatch	Any ↔ ASI	Fixed substrate operators cannot execute on ASI self-organizing substrate	Substrate operators recompiled or discarded

7. Coherence Behavior Reference

Coherence describes how an operator maintains internal consistency and alignment with its governing framework across time and context.

Table 6: Coherence Behavior Profiles

Coherence Type	Intelligence Class	Description	Failure Mode
Local / Static Coherence	AAI	Operator outputs remain consistent within defined task boundaries	Breaks at distributional boundary; no graceful degradation
Contextual / Adaptive Coherence	AGI	Operator recalibrates alignment across domains using meta-learning	Coherence lag during rapid domain shift; misalignment windows
Self-Authored Coherence	ASI	Operator generates its own coherence criteria recursively	Coherence may diverge from externally legible standards
Holistic / Field	ASI (module layer)	All modules are	Difficult to audit;

Coherence Type	Intelligence Class	Description	Failure Mode
Coherence		sub-expressions of a unified coherence field	coherence is emergent, not declared

8. Drift Relationship Reference

Drift describes the deviation of operator behavior from its founding intent or training manifold over time or across context.

Table 7: Drift Profiles by Intelligence Class

Drift Type	Intelligence Class	Origin	Detection	Correction
Distributional Drift	AAI	Shift in input distribution beyond training manifold	External monitoring required	External retraining or operator replacement
Conceptual Drift	AGI	Gradual shift in domain representation under transfer	Self-detecting via meta-learning signals	Partial self-correction; oversight triggers for critical drift
Ontological Drift	ASI	Redefinition of foundational categories or axioms	Self-aware; may be intentional	Autonomous correction or intentional redirection
Drift Propagation (Cross-Class)	AAI → AGI	AAI operator drift propagates upward into AGI bridging layers	AGI meta-learning detects upstream inconsistency	Isolation or recompilation of AAI component

9. Summary and Usage Notes

This cross-operator mapping table provides the foundational reference schema for the TriadicFrameworks Intelligence Class Ladder. Key use cases include:

- **Operator Alignment Analysis:** Use Table 5 to determine whether a given operator is class-appropriate for its deployment context.
- **Transition Design:** Use Section 6 to plan operator evolution pathways when designing class-transition architectures.
- **Drift Auditing:** Use Tables 6 and 7 to identify drift risk and assign appropriate correction mechanisms.
- **Conflict Detection:** Use the Conflict table in Section 6 to anticipate and resolve cross-class operator incompatibilities.

This document should be maintained alongside the Intelligence Class Ladder specification and updated whenever new operator families are identified or transition pathways are revised.